

“Instituto Politécnico Nacional”

“Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas”



Alumno: Ortiz Campos Kevin Daniel.

Grupo: 4TM3.

Escuela: Unidad Profesional Interdisciplinada en Ingeniería y Tecnologías Avanzadas.

Materia: Aplicaciones Distribuidas.

Profesor: Sierra Romero Noé.

Trabajo: Practica 3.

Practica 3 Práctica de Aplicaciones Distribuidas Servicios ResBul usando NodeJS

1. Crear una serie de servicios web usando un servidor NodeJS que realicen lo siguiente:
 - a. Requerimientos:
 - i. Todos los servicios reciben sus parámetros en una estructura JSON
 - ii. Todos los servicios responden en una estructura JSON
 - iii. Todos los servicios deben validar los parámetros y el resultado de la operación, e incluir un atributo en el JSON de respuesta que indique el resultado o un campo de error indicando el problema
 - b. Tareas por implementar
 - i. mascaracteres: recibe dos cadenas y regresa la que tenga más caracteres. Si son iguales, regresa la del primer parámetro
 - ii. menoscaracteres: recibe dos cadenas y regresa la que tenga menos caracteres. Si son iguales, regresa la del primer parámetro
 - iii. numcaracteres: recibe una cadena y regresa el número de caracteres que la cadena Dene
 - iv. palindroma: recibe una cadena y regresa true si la cadena es una palindroma, y false en caso contrario
 - v. concat: recibe dos cadenas y regresa la concatenación iniciando con el primer parámetro
 - vi. applysha256: recibe una cadena, le aplica una encriptación SHA256 y regresa como resultado la cadena original y la encriptada
 - vii. verifysha256: recibe una cadena encriptada, una cadena normal, a la cadena normal le aplica SHA256, la compara con la cadena encriptada y regresa true si coinciden, y false en otro caso.

2. Una vez finalizados, realiza las validaciones correspondientes en Postman
3. Documente sus códigos y pruebas
4. Suba sus evidencias en una carpeta de GitHub

Archivo: **Index.js**

```
const express = require('express');
const crypto = require('crypto');
const app = express();
const PORT = 3000;

// Middleware para procesar JSON en el cuerpo de la petición
app.use(express.json());

const sendResponse = (res, data, error = null) => {
  if (error) {
    return res.status(400).json({ status: "error", message: error });
  }
  return res.json({ status: "success", result: data });
};

// Mas caracteres
app.post('/mascaracteres', (req, res) => {
  const { str1, str2 } = req.body;
  if (typeof str1 !== 'string' || typeof str2 !== 'string')
    return sendResponse(res, null, "Se requieren dos cadenas: str1 y str2");

  const result = str2.length > str1.length ? str2 : str1;
  sendResponse(res, result);
});

// Menos caracteres
app.post('/menoscaracteres', (req, res) => {
  const { str1, str2 } = req.body;
  if (typeof str1 !== 'string' || typeof str2 !== 'string')
```

```
    return sendResponse(res, null, "Se requieren dos cadenas: str1 y str2");

    const result = str2.length < str1.length ? str2 : str1;
    sendResponse(res, result);
  });

  // Numero de caracteres
  app.post('/numcaracteres', (req, res) => {
    const { cadena } = req.body;
    if (typeof cadena !== 'string')
      return sendResponse(res, null, "Se requiere una cadena en el campo 'cadena'");

    sendResponse(res, cadena.length);
  });

  // Palíndroma
  app.post('/palindroma', (req, res) => {
    const { cadena } = req.body;
    if (typeof cadena !== 'string')
      return sendResponse(res, null, "Parametro invlido");

    const limpia = cadena.toLowerCase().replace(/[\W_]/g, "");
    const esPalindroma = limpia === limpia.split("").reverse().join("");
    sendResponse(res, esPalindroma);
  });

  // Concat
  app.post('/concat', (req, res) => {
    const { str1, str2 } = req.body;
    if (typeof str1 !== 'string' || typeof str2 !== 'string')
      return sendResponse(res, null, "Faltan cadenas para concatenar");

    sendResponse(res, str1 + str2);
  });

  // Apply SHA256
```

```
app.post('/applysha256', (req, res) => {
  const { cadena } = req.body;
  if (!cadena) return sendResponse(res, null, "Cadena vacía");

  const hash = crypto.createHash('sha256').update(cadena).digest('hex');
  sendResponse(res, { original: cadena, encriptada: hash });
});

// Verify SHA256
app.post('/verifysha256', (req, res) => {
  const { cadena_normal, cadena_encriptada } = req.body;
  if (!cadena_normal || !cadena_encriptada)
    return sendResponse(res, null, "Faltan parametros de comparacion");

  const hashGenerado =
crypto.createHash('sha256').update(cadena_normal).digest('hex');
  sendResponse(res, hashGenerado === cadena_encriptada);
});

app.listen(PORT, () => {
  console.log(` Servidor corriendo en http://localhost:${PORT} `);
});
```

Administrador: Símbolo del sistema - node index.js

```
C:\Windows\System32>cd D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto3_ResBul

C:\Windows\System32>D:

D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto3_ResBul>npm init -d
npm info using npm@11.6.2
npm info using node@v24.13.0
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (proyecto3_resbul)
version: (1.0.0)
description: Aprendiendo a usar funciones con metodo postman
entry point: (index.js)
test command:
git repository:
keywords:
author: Daniel Oc
license: (ISC)
type: (commonjs)
About to write to D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto3_ResBul\package.json:

{
  "name": "proyecto3_resbul",
  "version": "1.0.0",
  "description": "Aprendiendo a usar funciones con metodo postman",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Daniel Oc",
  "license": "ISC",
  "type": "commonjs"
}

Is this OK? (yes) yes
npm info ok

D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto3_ResBul>npm install express

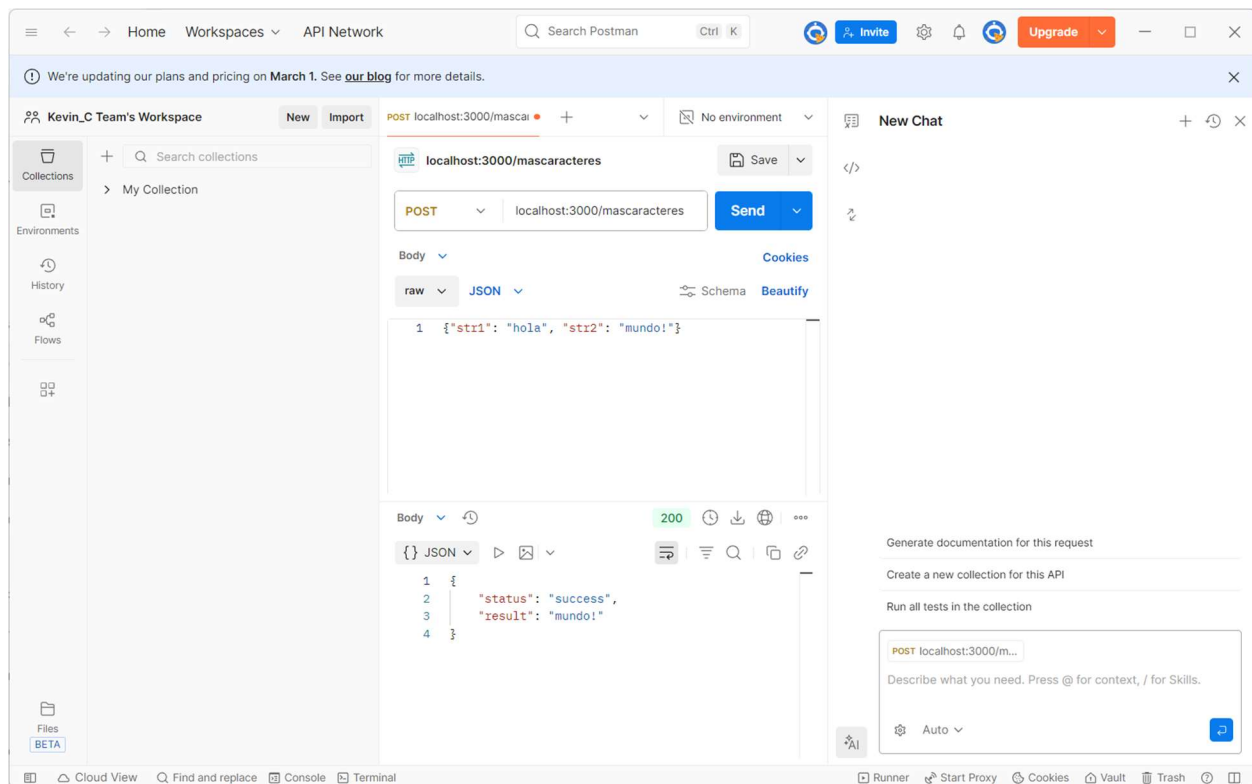
D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto3_ResBul>npm install express
added 65 packages, and audited 66 packages in 4s

22 packages are looking for funding
  run `npm fund` for details

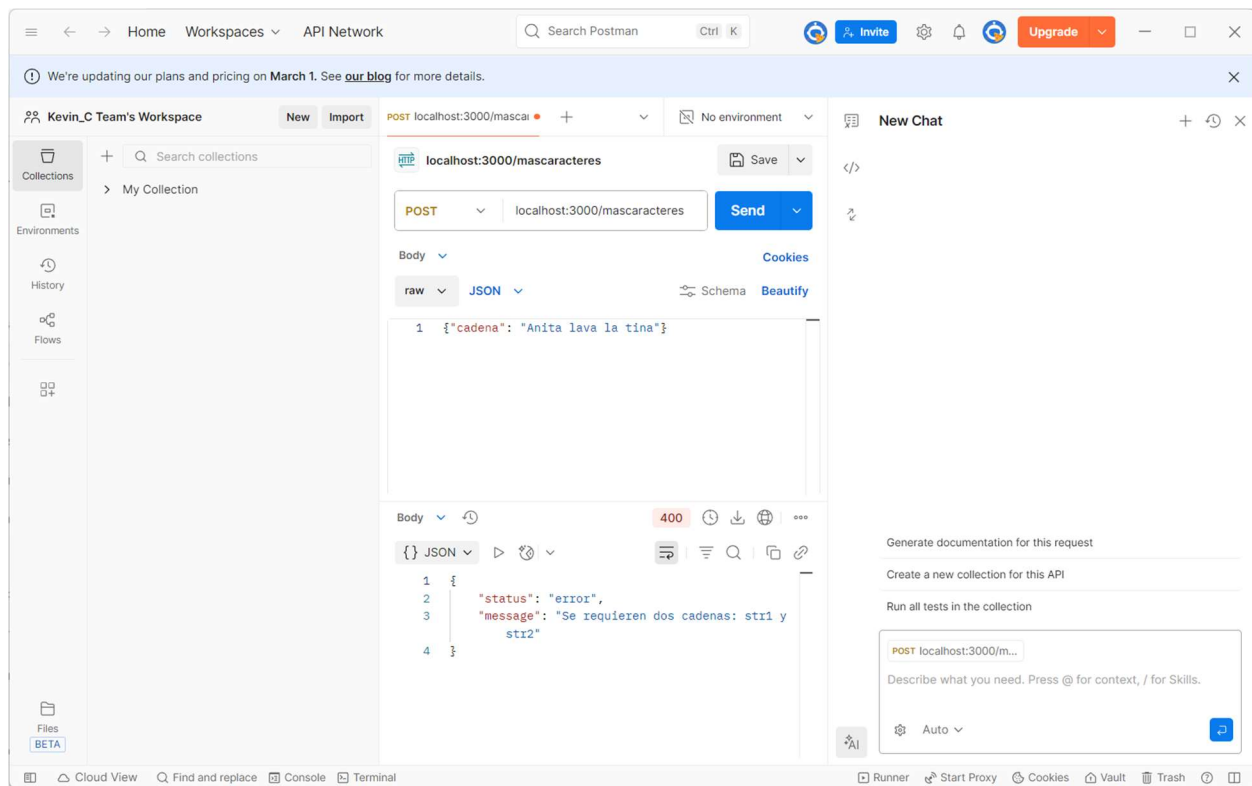
found 0 vulnerabilities

D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto3_ResBul>node index.js
Servidor corriendo en http://localhost:3000
```

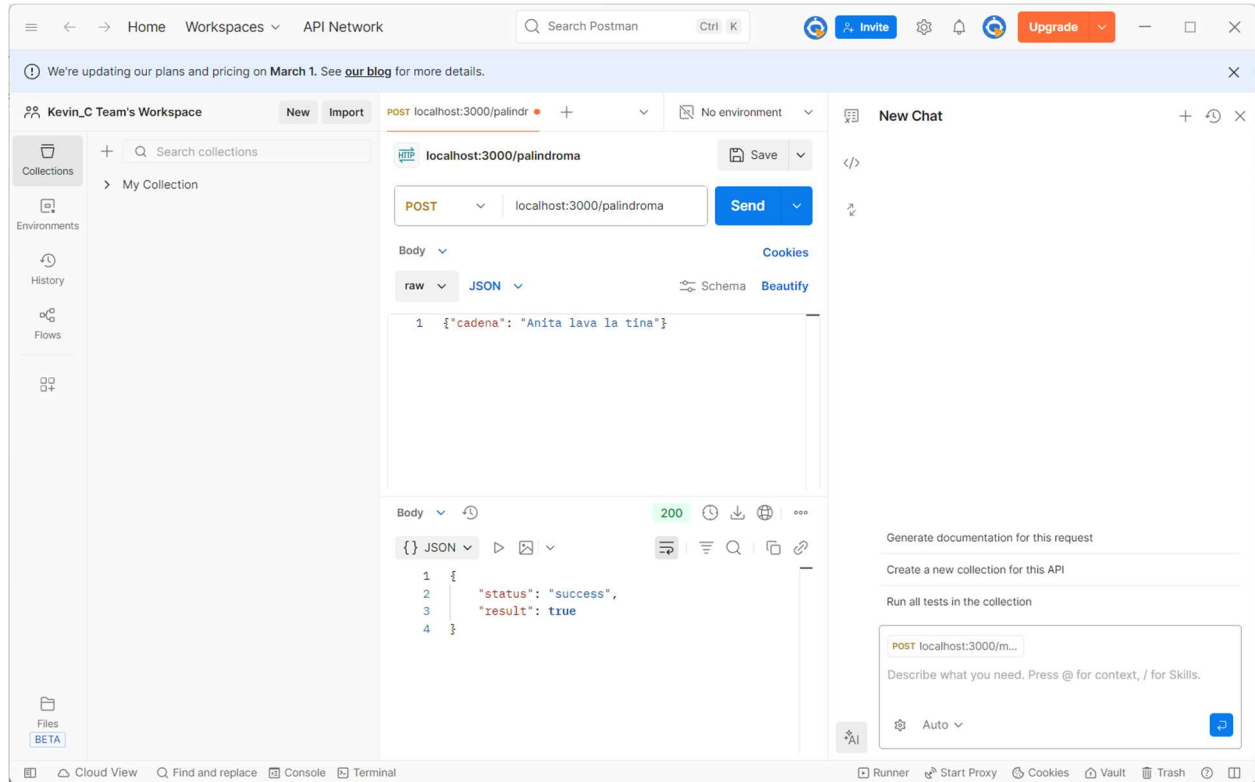
Prueba de mascaracteres en PostMan



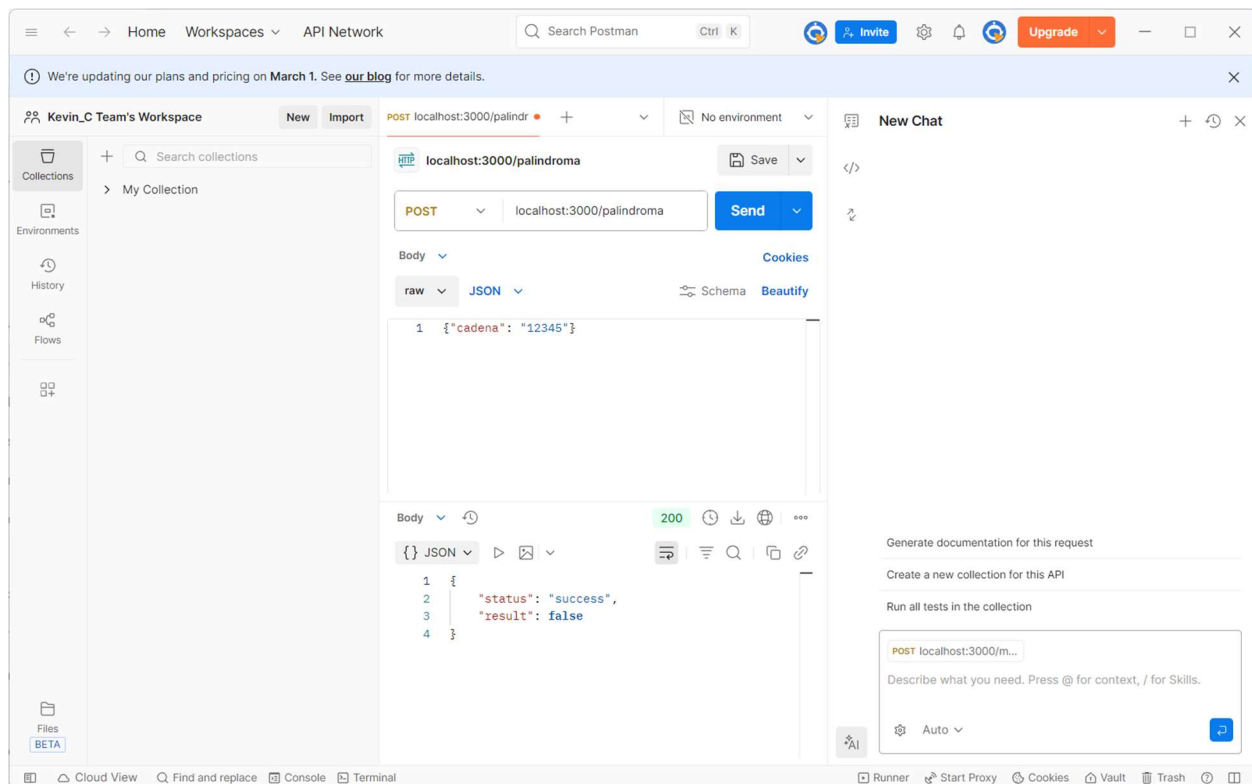
Prueba de mascaracteres en PostMan para que falle



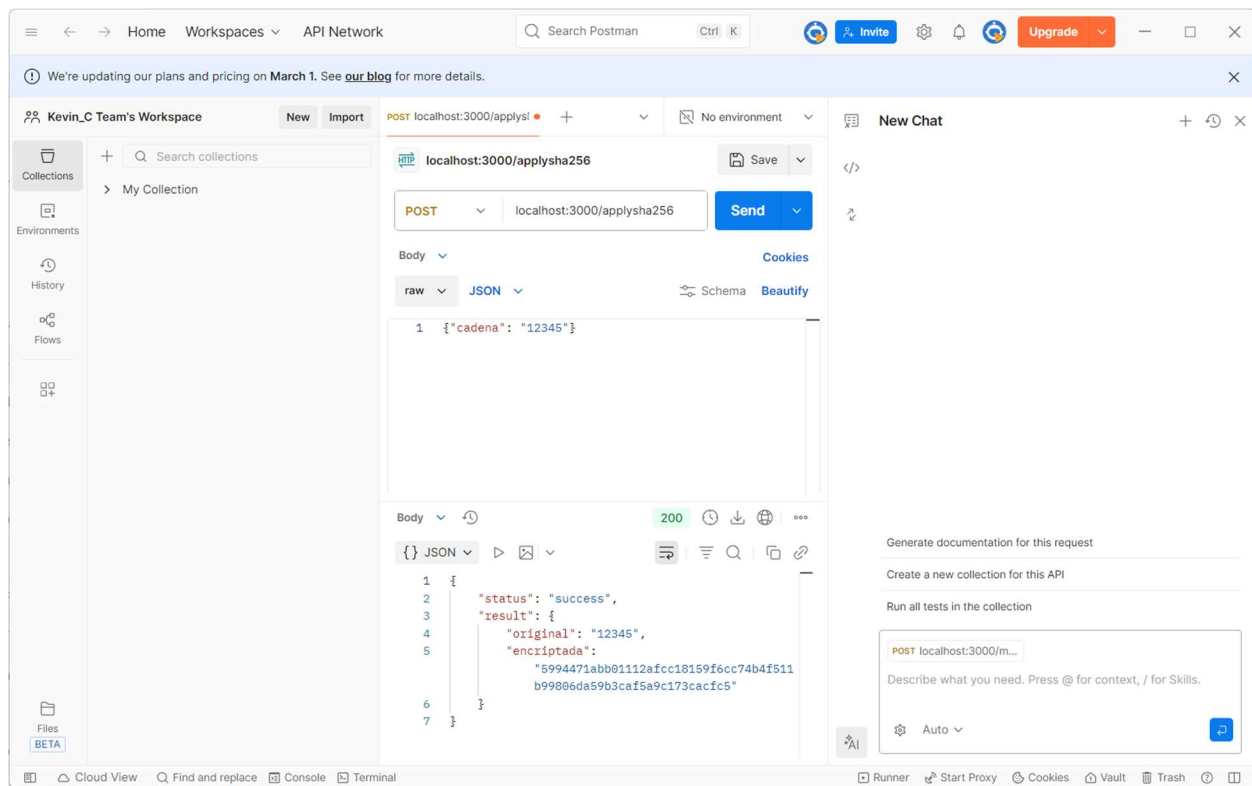
Prueba de palindroma en PostMan

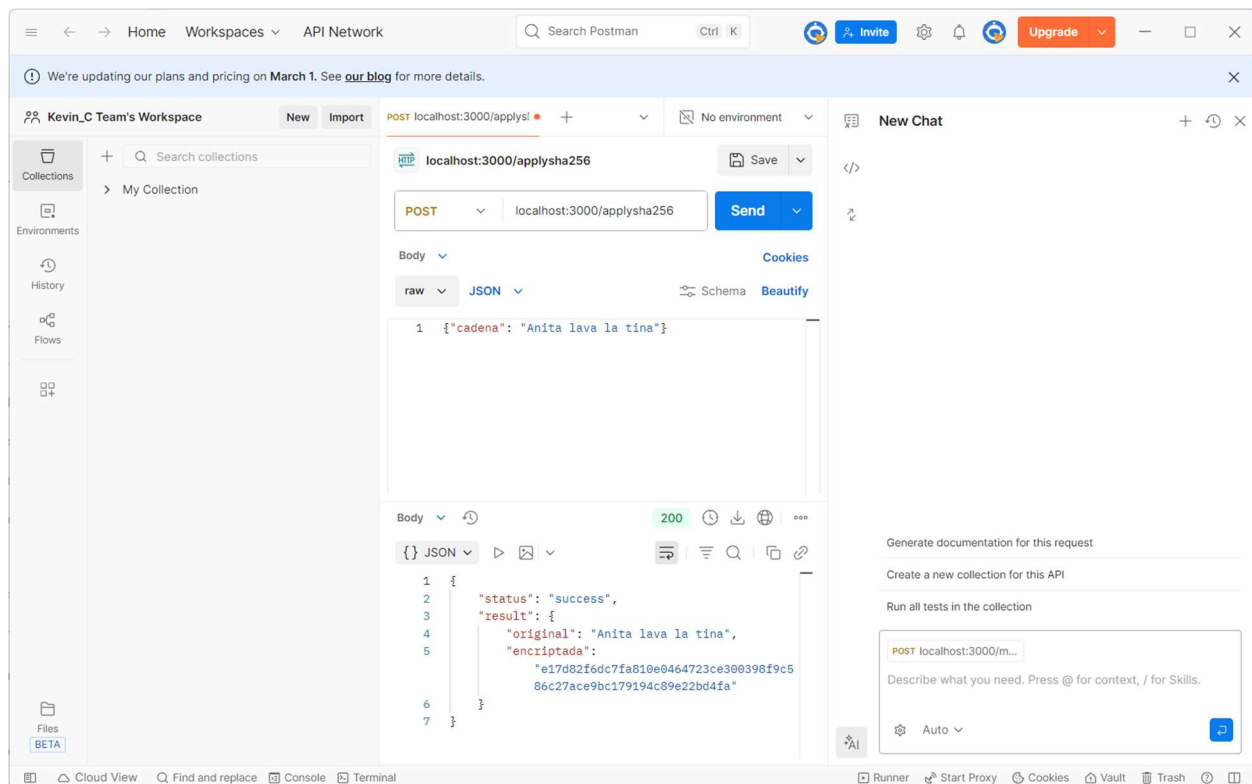


Prueba de palindroma en PostMan para que falle



Prueba de encriptacion en PostMan





Prueba de encriptacion en PostMan para que falle

